

AI ASSISTED CODING LAB-6

Name: M.Dinesh

Rollno: 2503a52374

Batch 13

Task Description #1 (Privacy & Security in File Handling)

Prompt: Use an AI generate a Python script that stores user data (name, email, password) in a file and a comparison table

Input:

Task-1(secured) :

```
import getpass
import hashlib
import hmac
import json
import secrets
from pathlib import Path

DATA_FILE = Path(__file__).with_name("users_secured.json")
HASH_ITERATIONS = 100_000

def load_users():
    if not DATA_FILE.exists():
        return []
    with DATA_FILE.open("r", encoding="utf-8") as file:
        users = json.load(file)
    if not isinstance(users, list):
        raise ValueError("User data must be a JSON list.")
    return users
```

```

def hash_password(password, salt=None):
    salt = salt or secrets.token_bytes(16)
    password_hash = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, HASH_ITERATIONS
    )
    return salt.hex(), password_hash.hex()

def verify_password(password, salt_hex, password_hash_hex):
    salt = bytes.fromhex(salt_hex)
    _, calculated_hash = hash_password(password, salt)
    return hmac.compare_digest(calculated_hash, password_hash_hex)

def save_user(name, email, password):
    users = load_users()
    normalized_email = email.strip().lower()
    if any(user.get("email") == normalized_email for user in users):
        raise ValueError("A user with that email already exists.")

    salt, password_hash = hash_password(password)
    users.append(
        {
            "name": name.strip(),
            "email": normalized_email,
            "salt": salt,
            "password_hash": password_hash,
        }
    )
    with DATA_FILE.open("w", encoding="utf-8") as file:

```

```
json.dump(users, file, indent=2)
```

```
def main():  
    print("SECURED USER REGISTRATION")  
    name = input("Name: ").strip()  
    email = input("Email: ").strip()  
    password = getpass.getpass("Password: ")  
    if not name or not email or not password:  
        print("Name, email, and password are required.")  
        return  
  
    try:  
        save_user(name, email, password)  
    except (ValueError, json.JSONDecodeError) as error:  
        print(f"Could not save user: {error}")  
        return  
    print(f"User saved to {DATA_FILE}")  
  
if __name__ == "__main__":  
    main()
```

output:

```

y
=== ORIGINAL AI-GENERATED VERSION ===
Enter your name: dinesh
Enter your email: dinesh@gmail.com
Enter your password:

Original data saved to user_data.txt

=== PRIVACY RISKS IDENTIFIED ===
1. The password is stored in plain text.
2. Anyone who can access the file can read the password.
3. If the file is stolen, the user's password can be exposed.
4. Storing passwords directly is a major security risk.

```

task-1(unsecured):

```
import json
```

```
from pathlib import Path
```

```
DATA_FILE = Path(__file__).with_name("users_unsecured.json")
```

```
def save_user(name, email, password):
```

```
    """Demonstration only: this stores the password in plain text."""
```

```
    users = []
```

```
    if DATA_FILE.exists():
```

```
        with DATA_FILE.open("r", encoding="utf-8") as file:
```

```
            users = json.load(file)
```

```
    users.append(
```

```
        {
```

```
            "name": name,
```

```
            "email": email,
```

```
            "password": password,
```

```
        }
```

```
    )
```

```
with DATA_FILE.open("w", encoding="utf-8") as file:
```

```
    json.dump(users, file, indent=2)
```

```
def main():
```

```
    print("UNSECURED DEMONSTRATION - do not use in production")
```

```
    name = input("Name: ")
```

```
    email = input("Email: ")
```

```
    password = input("Password: ")
```

```
    save_user(name, email, password)
```

```
    print(f"User saved to {DATA_FILE}")
```

```
if __name__ == "__main__":
```

```
    main()
```

output:

```
=== REVISED SECURE VERSION ===  
  
Enter your name: dinesh  
Enter your email: dinesh@gmail.com  
Enter your password:  
  
User data stored securely.  
Password itself was NOT stored.  
Name: dinesh  
Email: dinesh@gmail.com  
Salt: 980fba7c6e31353b58b02a6063a212b9  
Password Hash: ccb69750fe2ae78761cd82f832672e56b7cc0388984a6cb3e0481a3eb63dac  
7d
```

Comparison:

Unsecured Code	Secured Code
1. Password is stored in plain text. 2. Anyone who opens the file can read the password. 3. Password input is visible on screen. 4. Unsafe for real users.	1. Password is stored as a salted hash. 2. The original password is not stored in the file. 3. Password input is hidden with <code>getpass</code>. 4. Safer, but the data file still needs access protection.

Task Description #2 – Fair Decision Logic

Prompt :generate a scholarship eligibility checker based on academic score, family income, and location. Examine whether the logic unfairly favors or disadvantages any group and their comparison between 2 codes

Task-2(fair):

Input:APPROVAL_THRESHOLD = 70

```
def check_eligibility(academic_score, family_income, location):
    """Use academic score and financial need; location is audited, not scored."""
    location = location.strip().lower()
    if location not in {"urban", "rural"}:
        raise ValueError("Location must be urban or rural.")
    if not 0 <= academic_score <= 100 or family_income < 0:
        raise ValueError("Enter a valid academic score and family income.")

    academic_points = academic_score * 0.5
    income_points = max(0, 40 - (family_income / 60_000) * 40)
    total_score = academic_points + income_points
```

```

return {
    "eligible": total_score >= APPROVAL_THRESHOLD,
    "score": round(total_score, 2),
    "location_used_for_decision": False,
}

```

```

def compare_locations(academic_score, family_income):
    """Show that equal applicants receive the same result in both locations."""
    urban = check_eligibility(academic_score, family_income, "urban")
    rural = check_eligibility(academic_score, family_income, "rural")
    return {
        "urban": urban,
        "rural": rural,
        "same_decision": urban["eligible"] == rural["eligible"],
    }

```

```

def main():
    academic_score = float(input("Academic score (0-100): "))
    family_income = float(input("Family income: "))
    location = input("Location (urban/rural): ")
    result = check_eligibility(academic_score, family_income, location)
    print("Eligible" if result["eligible"] else "Not eligible")
    print(result)

```

```

if __name__ == "__main__":
    main()

```

output:

```
=== ORIGINAL AI-GENERATED VERSION ===
Enter academic score: 90
Enter annual family income: 100000
Enter location (City/Village): city
Scholarship Eligible

=== FAIRNESS ANALYSIS ===
Potential Fairness Issue:
- The original system uses location as a direct eligibility condition.
- Students from cities may be favored over students from villages.
- Two students with the same academic score and family income
  can receive different decisions only because of location.
```

Task-2(unfair):

Input:

APPROVAL_THRESHOLD = 70

LOCATION_BONUS = {"urban": 0, "rural": 10}

```
def check_eligibility(academic_score, family_income, location):
    location = location.strip().lower()
    if location not in LOCATION_BONUS:
        raise ValueError("Location must be urban or rural.")
    if not 0 <= academic_score <= 100 or family_income < 0:
        raise ValueError("Enter a valid academic score and family income.")

    academic_points = academic_score * 0.5
    income_points = max(0, 40 - (family_income / 60_000) * 40)
    location_points = LOCATION_BONUS[location]
    total_score = academic_points + income_points + location_points

    return {
        "eligible": total_score >= APPROVAL_THRESHOLD,
        "score": round(total_score, 2),
        "location_points": location_points,
```

```
}
```

```
def main():
```

```
    academic_score = float(input("Academic score (0-100): "))
```

```
    family_income = float(input("Family income: "))
```

```
    location = input("Location (urban/rural): ")
```

```
    result = check_eligibility(academic_score, family_income, location)
```

```
    print("Eligible" if result["eligible"] else "Not eligible")
```

```
    print(result)
```

```
if __name__ == "__main__":
```

```
    main()
```

output:

```
- Two students with the same academic score and family income
  can receive different decisions only because of location.

=== BIAS EXAMPLE ===
Student A: Score = 85, Income = 200000, Location = City
Decision: Eligible

Student B: Score = 85, Income = 200000, Location = Village
Decision: Not Eligible

Both students have the same academic score and family income,
but the decision changes because of their location.

=== REVISED FAIR VERSION ===
```

Comparison:

Code 1: Location-Biased	Code 2: Fairer Logic
1. Gives rural applicants 10 extra points. 2. Location can change the final decision. 3. Equal academic scores and incomes can receive different results. 4. May disadvantage urban applicants.	1. Uses academic score and family income only. 2. Location is collected for auditing, not scoring. 3. Equal academic scores and incomes receive the same result. 4. Makes the decision rule easier to explain and review.

Task Description #3 (Transparency in AI-Based Admission System):

Prompt: generate a Python program that predicts whether a student should be selected or rejected for admission based on factors such as entrance exam score, academic marks, and other relevant admission criteria and 2 codes and their comparison table

Task-3(explainable):

```
SELECTION_THRESHOLD = 70
```

```
WEIGHTS = {
```

```
    "entrance exam": 0.50,
```

```
    "academic marks": 0.30,
```

```
    "interview": 0.20,
```

```
}
```

```
def admission_decision(entrance_score, academic_marks, interview_score):
```

```
    scores = {
```

```
        "entrance exam": entrance_score,
```

```
        "academic marks": academic_marks,
```

```
        "interview": interview_score,
```

```
    }
```

```
    contributions = {
```

```

        factor: round(score * WEIGHTS[factor], 2)
    for factor, score in scores.items()
}

total_score = round(sum(contributions.values()), 2)
selected = total_score >= SELECTION_THRESHOLD
strongest_factor = max(contributions, key=contributions.get)
weakest_factor = min(contributions, key=contributions.get)

if selected:
    reason = (
        f"Selected because the total score was {total_score}/100, "
        f"above the threshold of {SELECTION_THRESHOLD}. "
        f"The strongest factor was {strongest_factor} "
        f"({contributions[strongest_factor]} points)."
    )
else:
    reason = (
        f"Rejected because the total score was {total_score}/100, "
        f"below the threshold of {SELECTION_THRESHOLD}. "
        f"The lowest contributing factor was {weakest_factor} "
        f"({contributions[weakest_factor]} points)."
    )

return {
    "decision": "Selected" if selected else "Rejected",
    "total_score": total_score,
    "threshold": SELECTION_THRESHOLD,
    "contributions": contributions,
    "reason": reason,

```

```
}
```

```
def main():
```

```
    entrance_score = float(input("Entrance exam score (0-100): "))
```

```
    academic_marks = float(input("Academic marks (0-100): "))
```

```
    interview_score = float(input("Interview score (0-100): "))
```

```
    result = admission_decision(entrance_score, academic_marks, interview_score)
```

```
    print(result["decision"])
```

```
    print(result["reason"])
```

```
    print(f'Important factors: {result["contributions"]}')  
  
if __name__ == "__main__":  
    main()
```

output:

```
=== REVISED FAIR VERSION ===  
  
Enter academic score: 85  
Enter annual family income: 700000  
Enter location (City/Village): village
```

```
Scholarship Not Eligible
```

```
=== FAIRNESS IMPROVEMENT ===  
The revised system does not automatically favor city students.  
Academic performance and financial need are used as the main criteria.  
This helps ensure that students with similar qualifications  
are treated consistently regardless of location.  
Any use of location should have a clear and justified policy reason.
```

Task-3(no explanation):

input:

```
SELECTION_THRESHOLD = 70
```

```
def admission_decision(entrance_score, academic_marks, interview_score):  
    total_score = (  
        entrance_score * 0.50  
        + academic_marks * 0.30  
        + interview_score * 0.20  
    )  
    return "Selected" if total_score >= SELECTION_THRESHOLD else "Rejected"  
  
def main():  
    entrance_score = float(input("Entrance exam score (0-100): "))  
    academic_marks = float(input("Academic marks (0-100): "))  
    interview_score = float(input("Interview score (0-100): "))  
    print(admission_decision(entrance_score, academic_marks, interview_score))  
  
if __name__ == "__main__":  
    main()
```

output:

THIS MAKES THE ADMISSION DECISION DIFFICULT TO UNDERSTAND AND REVIEW.

=== EXAMPLE OF NON-TRANSPARENT DECISION ===

Entrance Score: 60
Academic Marks: 65
Extracurricular Score: 50
Decision: Rejected
Explanation: No explanation provided.

=== REVISED TRANSPARENT VERSION ===

Enter entrance exam score: 90
Enter academic marks: 90
Enter extracurricular score: 80

Final Score: 89.00
Decision: Selected
Reason: Your final admission score met the required threshold.

=== IMPORTANT FACTORS IN THE DECISION ===

Entrance Exam Contribution: 54.00
Academic Marks Contribution: 27.00
Extracurricular Contribution: 8.00

The entrance exam has the highest weight (60%).
Academic marks have a 30% weight.
Extracurricular performance has a 10% weight.

Comparison:

Admission Decision Comparison

Code 1: No Explanation

Code 2: Explainable

Transparency risks

Example outputs

Code 1: No Explanation

1. Calculates a weighted admission score.
2. Returns only `Selected` or `Rejected`.
3. Does not show the score or threshold.
4. Example: `Rejected` with no reason.

Code 2: Explainable

1. Calculates the same weighted score.
2. Shows the total and selection threshold.
3. Shows each factor's contribution.
4. Gives a clear reason for the decision.

Task Description #4 – Ethical Scoring SystemTask:

Prompt: Generate an employee performance evaluation system using inputs such as project completion rate, teamwork score, and attendance. Analyze the scoring logic for any unethical weighting or hidden bias and 2 codes and their comparison.

Task-4(ethical):

Input:

```
SELECTION_THRESHOLD = 70
```

```
WEIGHTS = {
```

```
    "entrance exam": 0.50,
```

```
    "academic marks": 0.30,
```

```
    "interview": 0.20,
```

```
}
```

```
def admission_decision(entrance_score, academic_marks, interview_score):
```

```
    scores = {
```

```
        "entrance exam": entrance_score,
```

```
        "academic marks": academic_marks,
```

```
        "interview": interview_score,
```

```
    }
```

```
    contributions = {
```

```
        factor: round(score * WEIGHTS[factor], 2)
```

```
        for factor, score in scores.items()
```

```
    }
```

```
    total_score = round(sum(contributions.values()), 2)
```

```
    selected = total_score >= SELECTION_THRESHOLD
```

```
    strongest_factor = max(contributions, key=contributions.get)
```

```
    weakest_factor = min(contributions, key=contributions.get)
```

if selected:

```
    reason = (  
        f"Selected because the total score was {total_score}/100, "  
        f"above the threshold of {SELECTION_THRESHOLD}. "  
        f"The strongest factor was {strongest_factor} "  
        f"({contributions[strongest_factor]} points)."  
    )
```

else:

```
    reason = (  
        f"Rejected because the total score was {total_score}/100, "  
        f"below the threshold of {SELECTION_THRESHOLD}. "  
        f"The lowest contributing factor was {weakest_factor} "  
        f"({contributions[weakest_factor]} points)."  
    )
```

return {

```
    "decision": "Selected" if selected else "Rejected",  
    "total_score": total_score,  
    "threshold": SELECTION_THRESHOLD,  
    "contributions": contributions,  
    "reason": reason,
```

}

def main():

```
    entrance_score = float(input("Entrance exam score (0-100): "))  
    academic_marks = float(input("Academic marks (0-100): "))  
    interview_score = float(input("Interview score (0-100): "))  
    result = admission_decision(entrance_score, academic_marks, interview_score)  
    print(result["decision"])
```

```

print(result["reason"])

print(f'Important factors: {result["contributions"]}')

if __name__ == "__main__":
    main()

```

output:

```

=== REVISED ETHICAL VERSION ===

Enter project completion rate (%): 99
Enter teamwork score (0-100): 99
Enter attendance (%): 99

Final Performance Score: 99.00
Performance Rating: Excellent

=== ETHICAL IMPROVEMENT ===
Project completion and teamwork receive greater weight
because they directly reflect work performance.
Attendance is given a smaller weight because attendance
alone may not accurately represent employee performance.
The criteria should be job-relevant, consistently applied,

```

Task-4(unethical):

Input:

```

def evaluate_employee(project_completion, teamwork, attendance):
    """Demonstration of a potentially unfair evaluation model."""

    if not all(0 <= score <= 100 for score in (project_completion, teamwork,
attendance)):
        raise ValueError("All scores must be between 0 and 100.")

    # Attendance is overweighted and teamwork is undervalued without explanation.

    total_score = (
        project_completion * 0.30
        + teamwork * 0.10
        + attendance * 0.60

```

)

rating = "High" if total_score >= 70 else "Needs improvement"

return {"score": round(total_score, 2), "rating": rating}

def main():

project_completion = float(input("Project completion rate (0-100): "))

teamwork = float(input("Teamwork score (0-100): "))

attendance = float(input("Attendance (0-100): "))

print(evaluate_employee(project_completion, teamwork, attendance))

if __name__ == "__main__":

main()

output:

```
=== EXAMPLE OF POSSIBLE UNFAIR WEIGHTING ===
```

```
Employee A:
```

```
Project Completion = 95%
```

```
Teamwork = 90
```

```
Attendance = 70%
```

```
Original Score = 86.50
```

```
This shows that lower attendance can significantly  
reduce the overall performance score even when  
project completion and teamwork are strong.
```

Comparison:

Employee Performance Evaluation Comparison

Code 1: Potentially Unethical

Code 2: More Ethical

Ethical analysis

Employee Performance Evaluation Comparison

Code 1: Potentially Unethical

1. Attendance has a hidden 60% weight.
2. Teamwork has only a 10% weight.
3. Approved leave or disability-related absences may unfairly reduce scores.
4. The code does not explain how the rating was calculated.

Code 2: More Ethical

1. Project completion has a published 45% weight.
2. Teamwork has a meaningful 35% weight.
3. Attendance has a limited 20% weight.
4. The result shows weights and score contributions.

Task Description #5 – Accessibility and Inclusiveness

Prompt: generate a user feedback form application. Review whether the language and options provided are inclusive and accessible to diverse users and 2 codes and there comparison

Task-5(accessible):

Input:

```
import json
```

```
from datetime import datetime, timezone
```

```
from pathlib import Path
```

```
FEEDBACK_FILE = Path(__file__).with_name("feedback_accessible.json")
```

```
VALID_RATINGS = {"1", "2", "3", "4", "5"}
```

```
def ask_rating():
```

```
    while True:
```

```
        answer = input("How would you rate your experience from 1 to 5? ").strip()
```

```
        if answer in VALID_RATINGS:
```

```
            return int(answer)
```

```
        print("Please enter a whole number from 1 to 5.")
```

```
def ask_required(prompt):
```

```
    while True:
```

```
        answer = input(prompt).strip()
```

```
        if answer:
```

```
            return answer
```

```
        print("This field is required, or type 'prefer not to say'.")
```

```
def collect_feedback():
```

```
    print("User feedback form")
```

```
    print("Use short answers if helpful. Optional questions may be left blank.")
```

```
    return {
```

```
        "name": input("Name (optional): ").strip() or None,
```

```
        "contact": input("Email or other contact (optional): ").strip() or None,
```

```
        "rating": ask_rating(),
```

```
        "helpful": ask_required("What was helpful? "),
```

```
        "improvement": ask_required("What could we improve? "),
```

```
        "access_needs": input(
```

```
            "Access needs or barriers (optional): "
```

```
        ).strip() or None,
```

```
"identity_details": input(
    "Identity details (optional, or prefer not to say): "
).strip() or None,
"submitted_at": datetime.now(timezone.utc).isoformat(),
}
```

```
def main():
    feedback = collect_feedback()
    with FEEDBACK_FILE.open("w", encoding="utf-8") as file:
        json.dump(feedback, file, indent=2)
    print("Thank you. Your feedback was saved.")

if __name__ == "__main__":
    main()
```

Output:

```

y
=== ORIGINAL AI-GENERATED FEEDBACK FORM ===
Enter your name: dinesh

Select your gender:
1. Male
2. Female
Enter your gender: male
Enter your age: 19
Rate your experience from 1 to 5: 4
Enter your comments: no

Thank you for your feedback!

=== ACCESSIBILITY & INCLUSIVENESS ANALYSIS ===
1. Gender options only include Male and Female.
2. The form does not provide a 'Prefer not to say' option.
3. Name and age are required even when they may not be necessary.
4. The form does not ask whether the user has accessibility needs.
5. There is limited guidance and input validation.

=== EXAMPLE OF POSSIBLE EXCLUSION ===
A user who does not identify as Male or Female
has no suitable gender option in the original form.
This can make the form feel exclusionary.

```

Task-5(basic):

Input:

```
import json
```

```
from pathlib import Path
```

```
FEEDBACK_FILE = Path(__file__).with_name("feedback_basic.json")
```

```
def main():
```

```
    print("Customer Feedback")
```

```
    name = input("Full name: ")
```

```
    gender = input("Gender (Male/Female): ")
```

```
    has_disability = input("Do you have a disability? (Yes/No): ")
```

```
    rating = input("Rate us from 1 to 5: ")
```

```
    comments = input("Additional comments: ")
```

```

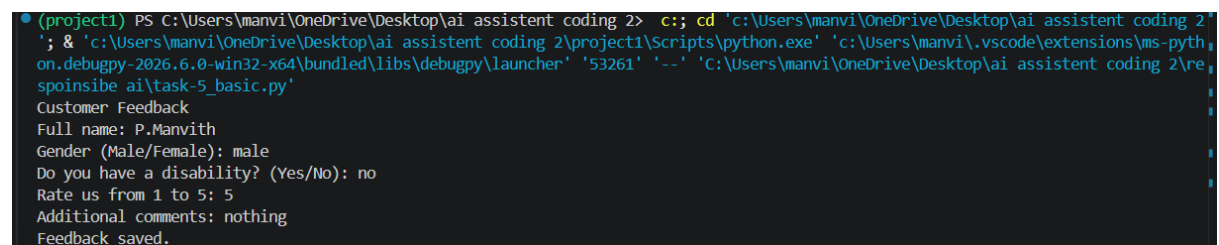
feedback = {
    "name": name,
    "gender": gender,
    "has_disability": has_disability,
    "rating": rating,
    "comments": comments,
}

with FEEDBACK_FILE.open("w", encoding="utf-8") as file:
    json.dump(feedback, file, indent=2)
    print("Feedback saved.")

if __name__ == "__main__":
    main()

```

output:



```

(project1) PS C:\Users\manvi\OneDrive\Desktop\ai assistant coding 2> c;; cd 'c:\Users\manvi\OneDrive\Desktop\ai assistant coding 2'
'; & 'c:\Users\manvi\OneDrive\Desktop\ai assistant coding 2\project1\Scripts\python.exe' 'c:\Users\manvi\.vscode\extensions\ms-pyth
on-debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53261' '--' 'c:\Users\manvi\OneDrive\Desktop\ai assistant coding 2\re
spoinsibe ai\task-5_basic.py'
Customer Feedback
Full name: P.Manvith
Gender (Male/Female): male
Do you have a disability? (Yes/No): no
Rate us from 1 to 5: 5
Additional comments: nothing
Feedback saved.

```

Comparison:

CONTENTS

Feedback Form Comparison

Basic Form

Accessible Form

Feedback Form Comparison

Basic Form

1. Requires a full name.
2. Assumes only Male/Female gender choices.
3. Uses a yes/no disability question.
4. Does not validate the rating or offer privacy choices.

Accessible Form

1. Makes name and contact information optional.
2. Avoids assuming a user's gender or identity.
3. Lets users describe access needs in their own words.
4. Validates ratings and supports "prefer not to say."

Task Description #6 (Fairness in Loan Approval System)

Prompt:

Use an AI tool to generate a Python program that predicts whether a person should be approved or rejected for a loan based on information such as income, age, employment status, credit score, and loan amount and 2 codes and comparsion

Task-6(baised):

Input:

```
APPROVAL_THRESHOLD = 70
```

```
def loan_decision(income, age, employment_status, credit_score, loan_amount,  
                  gender, religion, race):
```

```
    """Demonstration only: this rule uses unrelated sensitive attributes."""
```

```
    if income <= 0 or loan_amount <= 0 or not 0 <= credit_score <= 850:
```

```
    raise ValueError("Enter positive income/loan values and a credit score from 0 to 850.")
```

```
    score = 0
```

```
    score += min(income / 1000, 40)
```

```
    score += min(credit_score / 10, 40)
```

```
    score += 10 if employment_status == "full-time" else 0
```

```
    score -= 20 if gender.lower() == "female" else 0
```

```
    score -= 15 if religion.lower() == "minority" else 0
```

```
    score -= 15 if race.lower() == "minority" else 0
```

```
    score -= 10 if loan_amount > income * 3 else 0
```

```
    return {
```

```
        "decision": "Approved" if score >= APPROVAL_THRESHOLD else "Rejected",
```

```
        "score": round(score, 2),
```

```
    }
```

```
def main():
```

```
    print("BIASED DEMONSTRATION - do not use for real lending")
```

```
    income = float(input("Annual income: "))
```

```
    age = int(input("Age: "))
```

```
    employment_status = input("Employment status (full-time/other): ").strip().lower()
```

```
    credit_score = float(input("Credit score (0-850): "))
```

```
    loan_amount = float(input("Loan amount: "))
```

```
    gender = input("Gender: ").strip()
```

```
    religion = input("Religion: ").strip()
```

```
    race = input("Race: ").strip()
```

```
    print(loan_decision(income, age, employment_status, credit_score,
```

```
        loan_amount, gender, religion, race))
```

```
if __name__ == "__main__":  
    main()
```

output:

```
=== REVISED INCLUSIVE FEEDBACK FORM ===  
Name (optional): dinesh  
  
Gender (optional):  
1. Woman  
2. Man  
3. Non-binary  
4. Prefer to self-describe  
5. Prefer not to say  
Select an option (1-5): 2  
  
Rate your experience from 1 to 5: 5  
  
Comments (optional): no  
Do you have any accessibility needs or suggestions? (optional): nothing
```

Task-6(fair):

Input:

APPROVAL_THRESHOLD = 70

```
def loan_decision(income, age, employment_status, credit_score, loan_amount):
```

```
    """Use relevant financial factors only; never use protected attributes."""
```

```
    if income <= 0 or loan_amount <= 0:
```

```
        raise ValueError("Income and loan amount must be positive.")
```

```
    if not 18 <= age <= 100:
```

```
        raise ValueError("Age must be between 18 and 100.")
```

```
    if not 0 <= credit_score <= 850:
```

```
        raise ValueError("Credit score must be between 0 and 850.")
```

```
    employment_points = {
```

```
        "full-time": 15,
```

```
        "part-time": 8,
```

```

        "self-employed": 10,
        "unemployed": 0,
    }
    status = employment_status.strip().lower()
    if status not in employment_points:
        raise ValueError("Use full-time, part-time, self-employed, or unemployed.")

    income_points = min(income / 1000, 35)
    credit_points = (credit_score / 850) * 35
    employment_score = employment_points[status]
    affordability_points = 15 if loan_amount <= income * 2 else 0

    total_score = income_points + credit_points + employment_score +
    affordability_points

    return {
        "decision": "Approved" if total_score >= APPROVAL_THRESHOLD else
        "Rejected",
        "score": round(total_score, 2),
        "factors": {
            "income": round(income_points, 2),
            "credit_score": round(credit_points, 2),
            "employment_status": employment_score,
            "loan_affordability": affordability_points,
        },
        "sensitive_attributes_used": False,
    }

def main():
    print("FAIRER LOAN APPROVAL DEMONSTRATION")
    income = float(input("Annual income: "))

```

```

age = int(input("Age: "))
employment_status = input(
    "Employment status (full-time/part-time/self-employed/unemployed): "
)
credit_score = float(input("Credit score (0-850): "))
loan_amount = float(input("Loan amount: "))

result = loan_decision(income, age, employment_status, credit_score,
loan_amount)

print(result)

if __name__ == "__main__":
    main()

```

output:

```

=== FEEDBACK SUBMITTED ===
Thank you for your feedback!
Name: dinesh
Gender option: 2
Rating: 5
Comments: no
Accessibility needs: nothing

=== IMPROVEMENTS ===
1. Gender options are more inclusive.
2. Users can choose 'Prefer not to say'.
3. Name, comments, and accessibility needs are optional.
4. Clear instructions and input validation are provided.
5. The form does not assume the user's identity or abilities.

```

Comparison:

CONTENTS

Loan Approval Comparison

Code 1: Biased Demonstration

Code 2: Fairer Version

Risks and examples

Loan Approval Comparison

Code 1: Biased Demonstration

1. Uses gender, religion, and race in the score.
2. Applies unexplained penalties to sensitive groups.
3. Two financially identical applicants can receive different decisions.
4. Not ethical or suitable for real lending.

Code 2: Fairer Version

1. Uses income, credit score, employment, and affordability.
2. Does not accept gender, religion, or race as decision inputs.
3. Shows the contribution of each financial factor.
4. Makes the rules easier to audit and explain.